

Часть 1. Проверка гипотезы в Python и составление аналитической записки

Анализ продуктовых метрик сервиса Яндекс.Книги и проверка гипотезы о среднем времени активности пользователей

- Автор: Макковеева Елена
- Дата: 13.06.2025

Цели и задачи проекта

Цель проекта - анализ продуктовых метрик сервиса Яндекс.Книги и проверка гипотезы о связи лояльности пользователей и времени чтения/прослушивания книг. Предполагается, что пользователи с большим показателем LTV (Lifetime Value) более активны и проводят больше времени за потреблением контента.

Анализ продуктовых метрик сервиса Яндекс. Книги

- 2.1.1 Расчет MAU авторов
- 2.1.2 Расчет MAU произведений
- 2.1.3 Расчет Retention Rate
- 2.1.4 Расчет LTV
- 2.1.5 Расчет средней выручки прослушанного часа

Проверка гипотезы о среднем времени активности пользователей

- 2.2.1 Подготовка данных для анализа
- 2.2.2 Загрузка данных и знакомство с ними
- 2.2.3 Проверка гипотезы
- 2.2.4 Интерпретация результатов и подготовка аналитической записки.

Описание данных

Для анализа предоставлены данные сервиса Яндекс.Книги с 1 сентября по 11 декабря 2024 года, которые включают информацию о пользователях, платформах, времени, длительности сессий и типах контента.

Данные включают в себя несколько таблиц.

Таблица `bookmate.audition` содержит данные об активности пользователей и состоит из следующих полей:

- `audition_id` — уникальный идентификатор сессии чтения или прослушивания;
- `ruid` — идентификатор пользователя;
- `usage_platform_ru` — название платформы, с помощью которой пользователь слушал контент;
- `msk_business_dt_str` — дата события в формате строки (московское время);
- `app_version` — версия приложения, которая использовалась для чтения или прослушивания;
- `adult_content_flg` — был ли это контент для взрослых: True или False;
- `hours` — длительность чтения или прослушивания в часах;
- `hours_sessions_long` — продолжительность длинных сессий чтения или прослушивания в часах;
- `kids_content_flg` — был ли это детский контент: True или False;
- `main_content_id` — идентификатор основного контента;
- `usage_geo_id` — идентификатор географического местоположения.

Таблица `bookmate.content` содержит данные о контенте и состоит из следующих полей:

- `main_content_id` — идентификатор основного контента;
- `main_author_id` — идентификатор основного автора контента;
- `main_content_type` — тип контента;
- `main_content_name` — название контента;
- `main_content_duration_hours` — длительность контента в часах;
- `published_topic_title_list` — список жанров контента.

Таблица `bookmate.author` содержит данные об авторах контента и состоит из следующих полей:

- `main_author_id` — идентификатор основного автора контента;
- `main_author_name` — имя основного автора контента.

Таблица `bookmate.geo` содержит данные о местоположении и состоит из следующих полей:

- `usage_geo_id` — идентификатор географического положения;
- `usage_geo_id_name` — город или регион географического положения;

- usage_country_name — страна географического положения.

Содержимое проекта

С помощью SQL запросов были рассчитаны основные продуктовые метрики.

MAU авторов - рассчитано MAU топ-3 авторов сервиса в ноябре:

- Андрей Усачев 7107 уникальных пользователей в месяц;
- Лиана Шнайдер - 3338;
- Игорь Носов - 3063.

MAU произведений - рассчитано топ-3 популярных произведения в ноябре:

- Собачка Соня на даче, А. Усачев - 4597 уникальных пользователей в месяц;
- Женькин клад и другие школьные рассказы, И.Носов - 3050;
- Знаменитая собачка Соня, А.Усачев - 2785.

Retention Rate - расчет показал, что на 5 день активности в декабре Retention Rate составил 47%, это был минимальный показатель. Чаще всего пользователи возвращаются в первый день, RR составляет 63%. В остальные дни держится на уровне 50-55%.

LTV. Был проведен расчет метрики для пользователей из Москвы и Санкт-Петербурга.

В Санкт-Петербурге 12559 пользователей и LTV составляет 731,82 рубля, а в Москве 16808 пользователей и LTV составляет 764,55 рублей. В Москве пользователей больше и их средний LTV выше.

Средняя выручка за час активности - с сентября по ноябрь средняя выручка за час активности упала. В сентябре она составляла 61 рубль 70 копеек, а в ноябре 51 рубль 4 копейки.

Изучив данные, полученные при анализе LTV была выдвинута гипотеза. Она заключается в том, что пользователи, которые приносят больший LTV, являются более лояльными, а значит, проводят в среднем больше времени за чтением и прослушиванием книг в приложении. Предлагается сравнить пользователей из Санкт-Петербурга и Москвы, ведь при расчётах среднего LTV было определено, что эта метрика у петербуржцев ниже.

Нулевая гипотеза H_0 : $\mu_{СПб} \leq \mu_{Москва}$

Среднее время активности пользователей в Санкт-Петербурге не больше, чем в Москве.

Альтернативная гипотеза H_1 : $\mu_{СПб} > \mu_{Москва}$

Среднее время активности пользователей в Санкт-Петербурге больше, и это различие статистически значимо.

Гипотеза будет проверена с помощью одностороннего теста Манна-Уитни.

1. Загрузка данных и знакомство с ними

Загружаем данные, отфильтрованные для пользователей из Москвы и Санкт-Петербурга с их активностью (суммой часов чтения и прослушивания) из файла `/datasets/yandex_knigi_data.csv` в датасет `df_books`.

```
In [1]: #Импортируем библиотеку pandas для анализа данных  
import pandas as pd
```

```
In [2]: #Сохраняем данные в датасет df_books  
df_books = pd.read_csv('https://code.s3.yandex.net/datasets/yandex_knigi_data.csv')
```

```
In [3]: #Выведм первые пять строк для ознакомления с датасетом  
df_books.head()
```

```
Out[3]:
```

	Unnamed: 0	city	puid	hours
0	0	Москва	9668	26.167776
1	1	Москва	16598	82.111217
2	2	Москва	80401	4.656906
3	3	Москва	140205	1.840556
4	4	Москва	248755	151.326434

```
In [4]: #Выведем информацию о датасете  
df_books.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 8784 entries, 0 to 8783  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   Unnamed: 0   8784 non-null   int64  
1   city         8784 non-null   object  
2   puid         8784 non-null   int64  
3   hours        8784 non-null   float64
```

```
dtypes: float64(1), int64(2), object(1)
memory usage: 274.6+ KB
```

В датасете `df_books` 8784 строк и 4 столбца. Пропусков в столбцах не обнаружено. Типы данных соответствуют содержимому датасета.

Проверим датасет на дубликаты.

```
In [5]: #Проверяем дубликаты
df_books.duplicated().sum()
```

```
Out[5]: 0
```

Полных дубликатов не обнаружено.

Также проверим на дубликаты столбец с id пользователей `puid`.

```
In [6]: #Проверяем дубликаты id пользователей
df_books.duplicated(subset=['puid']).sum()
```

```
Out[6]: 244
```

Ознакомимся со списком `puid`, которые дублируются. Для удобства анализа, сразу отсортируем данные по столбцу `puid` и сохраним в датасет `df_books`.

```
In [7]: #Сортируем по id пользователей
df_books = df_books.sort_values(by='puid')
```

```
In [8]: #Выводим примеры дубликатов
df_books[df_books.duplicated(subset=['puid'], keep = False)].head(10)
```

```
Out[8]:
```

	Unnamed: 0	city	puid	hours
	35	Москва	2637041	10.317371
6247	6247	Санкт-Петербург	2637041	3.883926
6274	6274	Санкт-Петербург	9979490	1.302997
134	134	Москва	9979490	32.415573
145	145	Москва	10597984	42.931506

	Unnamed: 0	city	puid	hours
6279	6279	Санкт-Петербург	10597984	9.041320
6283	6283	Санкт-Петербург	10815097	0.323291
150	150	Москва	10815097	9.086655
6300	6300	Санкт-Петербург	13626259	1.648434
187	187	Москва	13626259	21.104167

Дубликаты действительно существуют, один id пользователя встречается и в Москве, и в Санкт-Петербурге. Таких дубликатов найден 244 строки. Удалим дублирующие строки, чтобы выборки стали независимы и сохраним обновленный датасет `df_books` .

```
In [9]: #Удаляем дубликаты
df_books=df_books.drop_duplicates(subset=['puid'])
```

```
In [10]: #Проверяем сколько осталось строк
df_books.shape[0]
```

```
Out[10]: 8540
```

После удаления дубликатов в датасете `df_books` осталось 8540 строк.

Проверим распределение пользователей по городам. Для этого сгруппируем данные по городу и посчитаем количество и долю пользователей.

```
In [11]: #Группируем по городам и считаем количество пользователей
df_books.groupby('city')['puid'].nunique()
```

```
Out[11]: city
Москва          6114
Санкт-Петербург  2426
Name: puid, dtype: int64
```

```
In [12]: #Группируем по городам и считаем доли пользователей
df_books['city'].value_counts(normalize=True).round(2)
```

```
Out[12]: Москва          0.72
Санкт-Петербург  0.28
Name: city, dtype: float64
```

В Москве 6114 пользователей или 72% от всех данных в датасете, в Санкт-Петербурге 2426 пользователей и 28%.

Сохраним данные о пользователях в городах в две выборки `moscow` и `spb`. Рассчитаем для этих выборок статистические показатели часов активности.

```
In [13]: #Сохраним данны о пользователях по городам
moscow = df_books[df_books['city']=='Москва']
spb = df_books[df_books['city']=='Санкт-Петербург']
```

```
In [14]: #Рассчитаем статистические показатели часов активности для пользователей из Москвы
moscow['hours'].describe().round(2)
```

```
Out[14]: count      6114.00
mean         10.85
std          36.92
min           0.00
25%           0.06
50%           0.90
75%           5.92
max          857.21
Name: hours, dtype: float64
```

```
In [15]: #Рассчитаем статистические показатели часов активности для пользователей из Санкт-Петербурга
spb['hours'].describe().round(2)
```

```
Out[15]: count      2426.00
mean         11.17
std          39.20
min           0.00
25%           0.07
50%           0.90
75%           6.18
max          978.76
Name: hours, dtype: float64
```

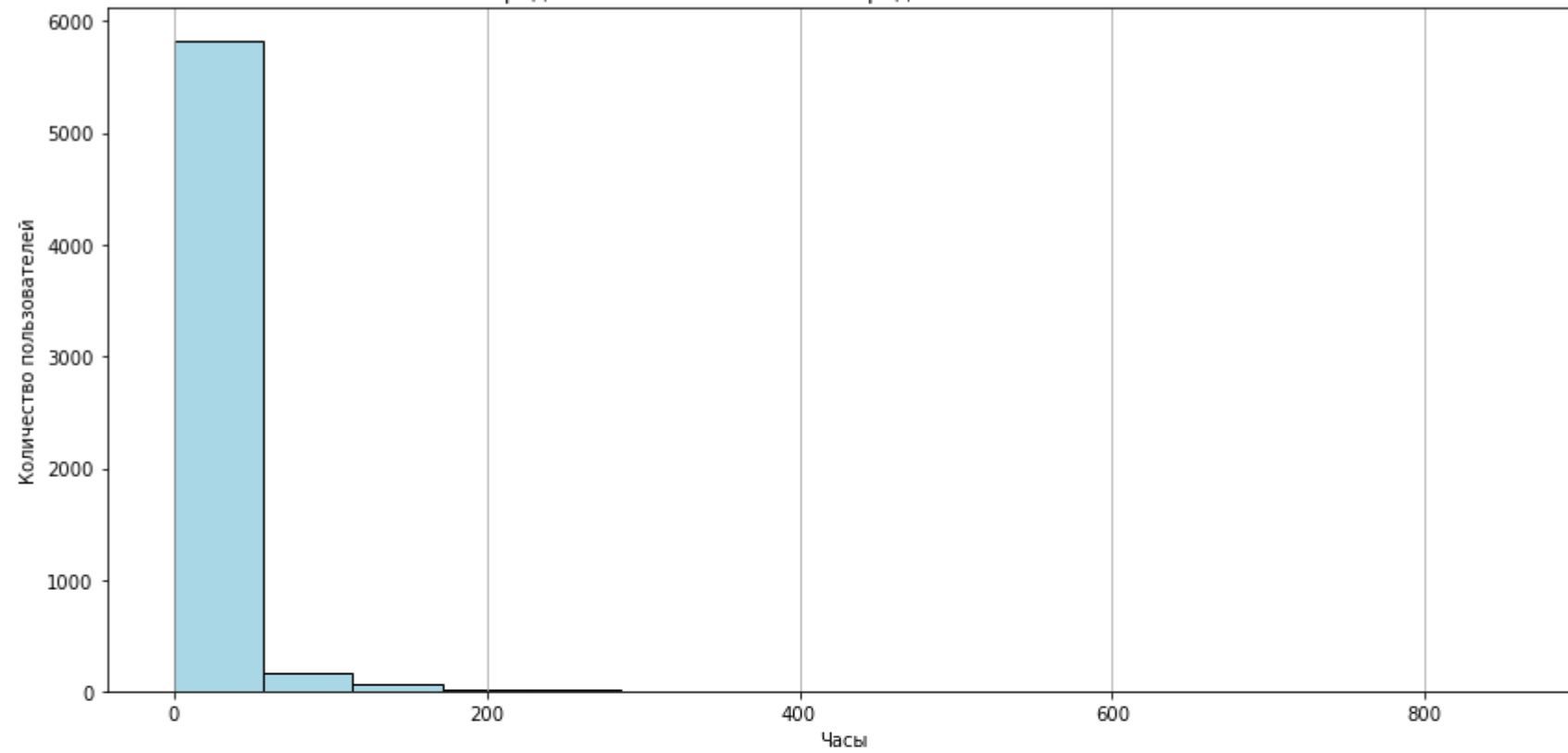
Анализ статистических показателей часов активности указывает на то, что распределение данных ассиметрично, так как стандартное отклонение (**39.2 для Санкт-Петербурга**) значительно больше среднего значения (**11.17 для Санкт-Петербурга**). Максимальные значения (**857.21 для Москвы**) превышают значения квантилей (**5.92 - 75 квантиль для Москвы**), что свидетельствует о наличии положительных выбросов. Также на наличие выбросов указывает значение медианы (**0.9 для Санкт-Петербурга, 0.9 для Москвы**), оно намного ниже среднего значения (**11.17 для Санкт-Петербурга, 10.85 для Москвы**).

Для подтверждения гипотезы об ассиметричном распределении данных, построим гистограммы распределения часов активности среди пользователей из Москвы и Санкт-Петербурга.

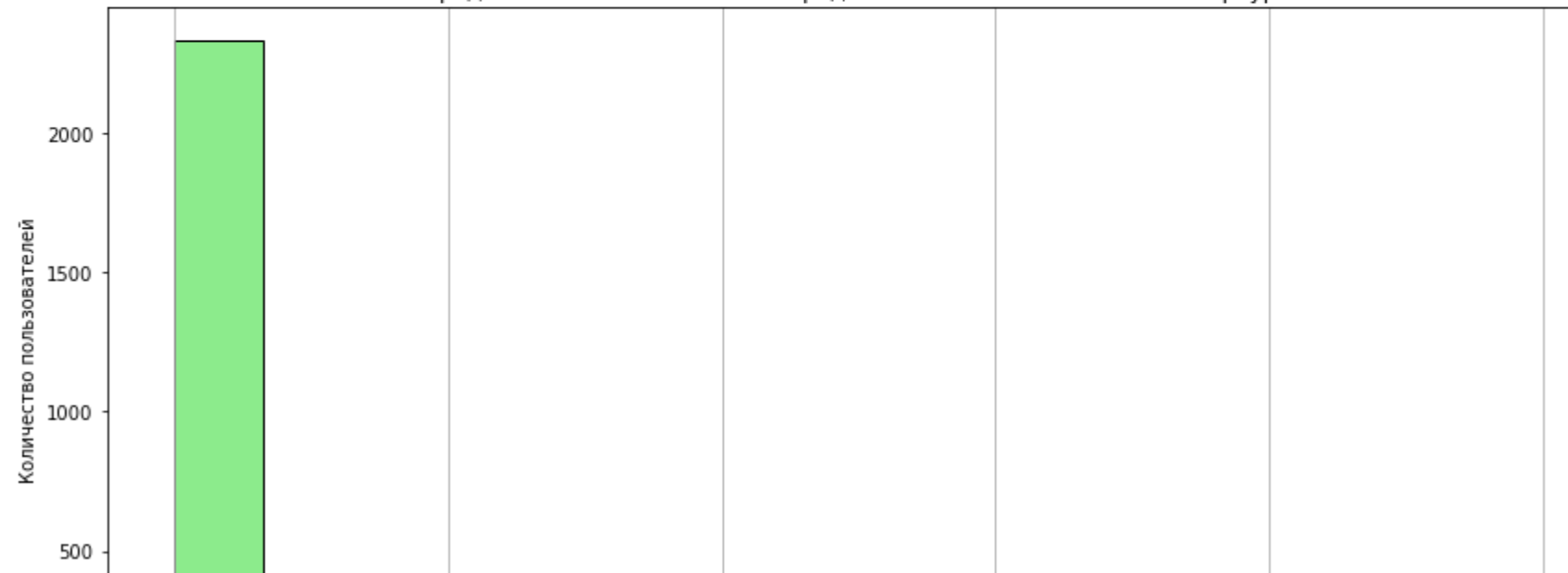
```
In [16]: #Импортируем библиотеку для визуализации данных  
import matplotlib.pyplot as plt
```

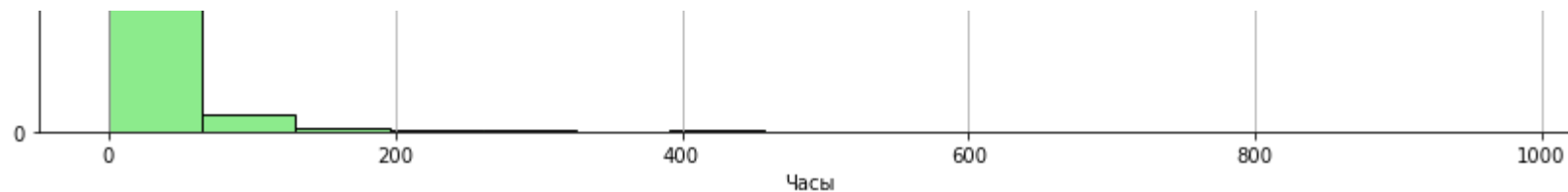
```
In [17]: #Создаем окно для двух графиков  
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(12, 12))  
  
#График для Москвы  
axes[0].hist(moscow['hours'], bins=15, color = 'lightblue', edgecolor='black')  
axes[0].set_title('Распределение часов активности среди пользователей из Москвы')  
axes[0].set_xlabel('Часы')  
axes[0].set_ylabel('Количество пользователей')  
axes[0].grid(True, axis='x')  
  
#График для Санкт-Петербурга  
axes[1].hist(spbf['hours'], bins=15, color = 'lightgreen', edgecolor='black')  
axes[1].set_title('Распределение часов активности среди пользователей из Санкт-Петербурга')  
axes[1].set_xlabel('Часы')  
axes[1].set_ylabel('Количество пользователей')  
axes[1].grid(True, axis='x')  
  
#Выводим два графика  
plt.tight_layout()  
plt.show()
```

Распределение часов активности среди пользователей из Москвы



Распределение часов активности среди пользователей из Санкт-Петербурга





С помощью гистограммы распределения мы видим, что у данных присутствует ассиметричное положительное распределение. **Большинство пользователей проводили до ~70 часов активности** в приложении, но есть пользователи с аномально большим количеством часов активности - более 200 часов за наблюдаемый период.

2. Проверка гипотезы о среднем времени активности пользователей

Была выдвинута гипотеза, что пользователи из Санкт-Петербурга проводят в среднем больше времени за чтением и прослушиванием книг в приложении, чем пользователи из Москвы.

- Нулевая гипотеза H_0 : Средняя активность пользователей в часах в двух группах (Москва и Санкт-Петербург) не различается.
- Альтернативная гипотеза H_1 : Средняя активность пользователей в Санкт-Петербурге больше, и это различие статистически значимо.

Так как данные распределены ассиметрично, а не нормально, то для проверки гипотезы используем тест Манна-Уитни. Уровень достоверности теста установим 95%.

```
In [18]: #Импортируем библиотеки для проведения теста
from scipy.stats import mannwhitneyu
import numpy as np
```

```
In [19]: #Устанавливаем уровень значимости
alpha = 0.05

mw_test = mannwhitneyu(
    moscow['hours'], #Выборка значений метрики для Москвы
    spb['hours'], #Выборка значений метрики для Санкт-Петербурга
    alternative='less' #Выбираем альтернативную гипотезу moscow < spb
)

if mw_test.pvalue > alpha:
    print(f'pvalue={mw_test.pvalue} > {alpha}')
    print('Нулевая гипотеза находит подтверждение! Средняя активность пользователей в часах в двух группах (Москва и Санкт-Петербург) не раз
else:
```

```
print(f'pvalue={mw_test.pvalue} < {alpha}')
```

```
print('Нулевая гипотеза не находит подтверждения! Средняя активность пользователей в Санкт-Петербурге больше, и это различие статистически значимо')
```

pvalue=0.27269236012414383 > 0.05

Нулевая гипотеза находит подтверждение! Средняя активность пользователей в часах в двух группах (Москва и Санкт-Петербург) не различается.

3. Аналитическая записка

Проведен тест Манна-Уитни для проверки гипотезы о том, что средняя активность пользователей в часах в Санкт-Петербурге выше, чем в Москве. Уровень статистической значимости выбран 5%, стандартное значение для анализа данных, с которым мы готовы допускать ошибки первого рода в эксперименте.

Тест показал результат **p-value равный 0.27**, что больше уровня значимости. Это означает, что вероятность случайного возникновения наблюдаемого различия велика. Соответственно, оснований для отказа от нулевой гипотезы нет. То есть имеющиеся данные не подтверждают утверждение, что активность пользователей в Санкт-Петербурге выше, чем в Москве.

Можно сказать, что **между LTV и активностью пользователей в Москве и Санкт-Петербурге отсутствует корреляция**. Более высокий LTV в Москве связан с большим количеством пользователей, чем в Санкт-Петербурге. Также пользователи могут покупать книги, но не слушать/читать их.

Часть 2. Анализ результатов A/B-тестирования

В интернет-магазине BitMotion Kit, в котором продаются геймифицированные товары для тех, кто ведет здоровый образ жизни, провели A/B тест. С помощью A/B теста было запланировано проверить как новый интерфейс сайта повлияет на конверсию в покупку.

Предыдущим аналитиком предоставлены данные по тесту.

Параметры теста:

- название теста: `interface_eu_test` ; группы: A (контрольная), B (новый интерфейс).

Таблица участников теста `ab_test_participants.csv` :

- `user_id` — идентификатор пользователя;
- `group` — группа пользователя;
- `ab_test` — название теста;
- `device` — устройство, с которого происходила регистрация.

Архив с одним csv-файлом, в котором собраны события 2020 года. Структура файла `ab_test_events.zip` :

- `user_id` — идентификатор пользователя;
- `event_dt` — дата и время события;
- `event_name` — тип события;
- `details` — дополнительные данные о событии.

1. Цели исследования

Цель исследования - оценить влияние новой версии сайта на увеличение количества пользователей, которые совершат покупку.

Для этого будет проведен A/B тест, в котором пользователи будут разделены на две группы:

- группа А пользователей, которые будут видеть старый интерфейс;
- группа В пользователей, которым будет показан новый интерфейс.

Сформировано две гипотезы:

- Нулевая: новый интерфейс не повлияет на конверсию в покупку;
- Альтернативная: новый интерфейс, в течение 7 дней после регистрации пользователя, увеличит конверсию в покупку на 3%.

2. Загрузка и проверка данных

Выгрузим данные по тесту и сохраним их. Данные об участниках теста в `participants` , события участников теста в `events` .

In [20]:

```
#Выгрузим данные по тесту
participants = pd.read_csv('https://code.s3.yandex.net/datasets/ab_test_participants.csv')
events = pd.read_csv('https://code.s3.yandex.net/datasets/ab_test_events.zip',
                     parse_dates=['event_dt'], low_memory=False)
```

In [21]:

```
#Выведем первые пять строк датасета participants
participants.head()
```

Out[21]:

	user_id	group	ab_test	device
0	0002CE61FF2C4011	B	interface_eu_test	Mac

	user_id	group	ab_test	device
1	001064FEAAB631A1	B	recommender_system_test	Android
2	001064FEAAB631A1	A	interface_eu_test	Android
3	0010A1C096941592	A	recommender_system_test	Android
4	001E72F50D1C48FA	A	interface_eu_test	Mac

```
In [22]: #Выведем информацию о датасете participants
participants.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14525 entries, 0 to 14524
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   user_id     14525 non-null   object
1   group       14525 non-null   object
2   ab_test     14525 non-null   object
3   device      14525 non-null   object
dtypes: object(4)
memory usage: 454.0+ KB
```

В датасете participants 14525 строк и 4 столбца, пропусков нет, типы данных установлены верно.

```
In [23]: #Выведем первые пять строк датасета events
events.head()
```

Out[23]:

	user_id	event_dt	event_name	details
0	GLOBAL	2020-12-01 00:00:00	End of Black Friday Ads Campaign	ZONE_CODE15
1	CCBE9E7E99F94A08	2020-12-01 00:00:11	registration	0.0
2	GLOBAL	2020-12-01 00:00:25	product_page	NaN
3	CCBE9E7E99F94A08	2020-12-01 00:00:33	login	NaN
4	CCBE9E7E99F94A08	2020-12-01 00:00:52	product_page	NaN

```
In [24]: #Выведем информацию о датасете events
events.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 787286 entries, 0 to 787285
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   user_id     787286 non-null  object
1   event_dt    787286 non-null  datetime64[ns]
2   event_name  787286 non-null  object
3   details     249022 non-null  object
dtypes: datetime64[ns](1), object(3)
memory usage: 24.0+ MB
```

В датасете `events` 787286 строк и 4 столбца. Пропуски только в столбце `details` и они не являются критичными для оценки результатов теста, поэтому их не нужно заполнять или удалять. Типы данных установлены верно.

```
In [25]: #Проверим датасет participants на дубликаты
participants.duplicated().sum()
```

```
Out[25]: 0
```

```
In [26]: #Проверим датасет events на дубликаты
events.duplicated().sum()
```

```
Out[26]: 36318
```

В датасете `participants` дубликаты не найдены, в `events` 36318 дубликатов. Удалим дублирующие строки.

```
In [27]: #Удалим дубликаты в датасете events
events = events.drop_duplicates()
```

```
In [28]: #Выведем информацию о датасете events после удаления дубликатов
events.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 750968 entries, 0 to 787285
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   user_id     750968 non-null  object
1   event_dt    750968 non-null  datetime64[ns]
2   event_name  750968 non-null  object
```

```
3 details      244905 non-null object
dtypes: datetime64[ns](1), object(3)
memory usage: 28.6+ MB
```

Теперь в датасете events 750968 строк.

3. Оценка корректности проведения теста

Для проведения оценки корректности теста, создадим отдельный датасет participants_test по тесту interface_eu_test , так как в первичном датасете participants собраны данные о разных тестах.

In [29]:

```
#Сохраним данные о проведенном тесте в датасете participants_test
participants_test = participants[participants['ab_test'] == 'interface_eu_test']
```

In [30]:

```
#Выведем информацию о датасете
participants_test.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 10850 entries, 0 to 14524
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype  
---  -
0   user_id     10850 non-null   object 
1   group       10850 non-null   object 
2   ab_test     10850 non-null   object 
3   device      10850 non-null   object 
dtypes: object(4)
memory usage: 423.8+ KB
```

В датасете participants 10850 строк.

In [31]:

```
#Проверим распределение пользователей по группам
participants_test['group'].value_counts(normalize=True).round(3)
```

Out[31]:

```
B    0.504
A    0.496
Name: group, dtype: float64
```

In [32]:

```
#Проверим пересечение пользователей в группах
intersection = list(set(participants_test[(participants_test['group']=='A')]) & set(participants_test[(participants_test['group']=='B')]))
intersection
```

```
Out[32]: ['user_id', 'group', 'ab_test', 'device']
```

Распределение пользователей по группам симметричное, в группе А 50.4% пользователей, в группе В 49.6% пользователей. Разница в количестве пользователей между группами 0,8%. Пересечения пользователей между группами не обнаружено.

Тест проведен корректно.

Объединим датасет `events` и `participants_test`, чтобы проанализировать действия пользователей из теста. Данные сохраним в новый датасет `df`.

```
In [33]: #Объединяем датасеты  
df = events.merge(participants_test, on = 'user_id', how = 'inner')
```

```
In [34]: #Выведем первые пять строк для проверки  
df.head()
```

```
Out[34]:
```

	user_id	event_dt	event_name	details	group	ab_test	device
0	5F506CEBEDC05D30	2020-12-06 14:10:01	registration	0.0	A	interface_eu_test	iPhone
1	5F506CEBEDC05D30	2020-12-07 01:25:14	login	NaN	A	interface_eu_test	iPhone
2	5F506CEBEDC05D30	2020-12-07 01:25:47	login	NaN	A	interface_eu_test	iPhone
3	5F506CEBEDC05D30	2020-12-09 12:40:49	login	NaN	A	interface_eu_test	iPhone
4	5F506CEBEDC05D30	2020-12-09 12:40:49	product_page	NaN	A	interface_eu_test	iPhone

```
In [35]: #Выведем информацию о датасете  
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
Int64Index: 73545 entries, 0 to 73544  
Data columns (total 7 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   user_id     73545 non-null  object  
1   event_dt    73545 non-null  datetime64[ns]  
2   event_name  73545 non-null  object  
3   details     20441 non-null  object  
4   group       73545 non-null  object  
5   ab_test     73545 non-null  object  
6   device      73545 non-null  object
```

```
dtypes: datetime64[ns](1), object(6)
memory usage: 4.5+ MB
```

В объединенном датасете `df` получилось 73545 строк событий, которые относятся к участникам теста `interface_eu_test`. Для дальнейшего анализа конверсии в покупку выведем все уникальные события из столбца `event_name`.

```
In [36]: #События, которые встречаются в столбце event_name
df['event_name'].unique()
```

```
Out[36]: array(['registration', 'login', 'product_page', 'product_cart',
               'purchase'], dtype=object)
```

Воронка продаж для пользователей интернет-магазина выглядит следующим образом: регистрация -> авторизация -> страница товара -> карточка товара -> покупка.

В альтернативной гипотезе было сказано, что новый интерфейс должен увеличить конверсию в покупку в течение 7 дней с момента регистрации. Поэтому нам необходимо отфильтровать действия пользователей, которые произошли в течение 7 дней. Для этого сначала мы ведем все события с регистрацией пользователей и сохраним их в `registration`.

```
In [37]: #Фильтруем только регистрацию пользователем
registration = df[df['event_name']=='registration'][['user_id', 'event_dt']]
registration.head()
```

```
Out[37]:
```

	user_id	event_dt
0	5F506CEBEDC05D30	2020-12-06 14:10:01
6	51278A006E918D97	2020-12-06 14:37:25
11	A0C1E8EFAD874D8B	2020-12-06 17:20:22
14	275A8D6254ACF530	2020-12-06 19:36:54
28	0B704EB2DC7FCA4B	2020-12-06 19:42:20

Теперь мы присоединим дату регистрации к основному датасету `df` по `id` пользователя. Чтобы не было конфликта в наименовании столбцов, столбец со всеми действиями пользователей по датам мы оставим как `event_dt`, а столбец с регистрацией назовем `event_dt_reg`.

```
In [38]: #Присоединяем даты регистрации к основному датасету
df = df.merge(registration, on = 'user_id', suffixes=('_', '_reg'))
df.head()
```

Out[38]:

		user_id	event_dt	event_name	details	group	ab_test	device	event_dt_reg
0	5F506CEBEDC05D30	2020-12-06 14:10:01	registration	0.0	A	interface_eu_test	iPhone	2020-12-06 14:10:01	
1	5F506CEBEDC05D30	2020-12-07 01:25:14	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	
2	5F506CEBEDC05D30	2020-12-07 01:25:47	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	
3	5F506CEBEDC05D30	2020-12-09 12:40:49	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	
4	5F506CEBEDC05D30	2020-12-09 12:40:49	product_page	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	

Посчитаем количество дней с момента регистрации и сохраним данные в новом столбце `days_since_reg` .

In [39]:

```
#Посчитаем количество дней с момента регистрации
df['days_since_reg'] = (pd.to_datetime(df['event_dt']) - pd.to_datetime(df['event_dt_reg'])).dt.days
```

In [40]:

```
#Выведем первые пять строк для проверки
df.head()
```

Out[40]:

		user_id	event_dt	event_name	details	group	ab_test	device	event_dt_reg	days_since_reg
0	5F506CEBEDC05D30	2020-12-06 14:10:01	registration	0.0	A	interface_eu_test	iPhone	2020-12-06 14:10:01	0	
1	5F506CEBEDC05D30	2020-12-07 01:25:14	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	0	
2	5F506CEBEDC05D30	2020-12-07 01:25:47	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	0	
3	5F506CEBEDC05D30	2020-12-09 12:40:49	login	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	2	
4	5F506CEBEDC05D30	2020-12-09 12:40:49	product_page	NaN	A	interface_eu_test	iPhone	2020-12-06 14:10:01	2	

Отфильтруем только те события, которые произошли в течение 7 дней после регистрации, и сохраним их в контрольный датасет `df_test` .

In [41]:

```
#Оставляем данные за первые 7 дней с момента регистрации
df_test = df[df['days_since_reg']<=7]
```

Оцените достаточность выборки для получения статистически значимых результатов А/В-теста. Заданные параметры:

- базовый показатель конверсии — 30%,
- мощность теста — 80%,

- достоверность теста — 95%.

Также для корректности проведения теста проанализируем достаточность выборки для получения статистически значимых результатов A/B теста.

Параметры для проверки:

- базовый показатель конверсии 30%
- ожидаем эффект +3% к базовой конверсии
- мощность теста 80%
- достоверность теста 95%.

In [42]:

```
import statsmodels.stats.api as sms

#Задаём параметры ожидаемого эффекта
effect_size = sms.proportion_effectsize(0.3, 0.33)

#Рассчитываем размер выборки
required_n = sms.NormalIndPower().solve_power(effect_size, power=0.8, alpha=0.05, ratio=1)

print(f'Необходимый размер выборки для каждой группы: {int(required_n)}')
```

Необходимый размер выборки для каждой группы: 3761

Необходимый размер выборки для каждой группы должен быть не менее 3761 пользователя. Проверим размер выборки в данных о проведенном тесте в датасете `df_test`.

In [43]:

```
print(f"Размер выборки группы A: {df_test[df_test['group'] == 'A']['user_id'].nunique()}")
print(f"Размер выборки группы B: {df_test[df_test['group'] == 'B']['user_id'].nunique()}")
```

Размер выборки группы A: 5383

Размер выборки группы B: 5467

Размеры выборки групп превышают рассчитанное значение в 3761 пользователя, поэтому можем считать, что результат теста будет надежным.

Для проведения теста рассчитаем общее количество пользователей (`total_users`), количество пользователей совершивших покупку (`buyers`) и конверсию по группам (`conversion`).

In [44]:

```
#Общее количество пользователей
total_users = df_test.groupby('group')['user_id'].nunique()
total_users
```

```
Out[44]: group
A      5383
B      5467
Name: user_id, dtype: int64
```

```
In [45]: #Количество пользователей, совершивших покупку
buyers = df_test[df_test['event_name']=='purchase'].groupby('group')['user_id'].nunique()
buyers
```

```
Out[45]: group
A      1542
B      1687
Name: user_id, dtype: int64
```

```
In [46]: #Конверсия в покупку между группами пользователей
conversion = buyers/total_users
conversion.round(2)
```

```
Out[46]: group
A      0.29
B      0.31
Name: user_id, dtype: float64
```

Предварительно, оценив полученные результаты общего количества пользователей, покупателей и конверсии, можно сказать, что в тестовой группе конверсия в покупку выросла на 2% пункта. Проверим изменение конверсии далее с помощью статистического теста.

4. A/B тест и оценка конверсии:

Для оценки изменения конверсии был выбран z-тест пропорций, так как метрика, которую проверяем является долевым (доля пользователей, совершивших покупку в течение семи дней после регистрации).

```
In [47]: from statsmodels.stats.proportion import proportions_ztest

#Рассчитаем размер группы A
n_a = total_users['A']

#Рассчитаем размер группы B
n_b = total_users['B']

#Рассчитаем количество покупок группы A
m_a = buyers['A']

#Рассчитаем количество покупок группы B
```

```

m_b = buyers['B']

#Рассчитаем доли покупок в каждой группе
p_a, p_b = m_a/n_a, m_b/n_b

print(f'Размер группы A - {n_a}, размер группы B - {n_b}, покупок в группе A - {m_a}, покупок в группе B - {m_b}')

#Проверяем предпосылку о достаточном количестве данных
if (p_a*n_a > 10)and((1-p_a)*n_a > 10)and(p_b*n_b > 10)and((1-p_b)*n_b > 10):
    print('Предпосылка о достаточном количестве данных выполняется!')
else:
    print('Предпосылка о достаточном количестве данных НЕ выполняется!')

#Устанавливаем уровень значимости
alpha = 0.05

#Проводим Z-тест пропорций
stat_ztest, p_value_ztest = proportions_ztest(
    [m_a, m_b],
    [n_a, n_b],
    alternative='smaller' #Альтернативная гипотеза p_a < p_b
)

#Выводим значение p_value
print(f'pvalue={p_value_ztest}')
print(f'stat_ztest={stat_ztest}')

#Формируем итоговый вывод
if p_value_ztest > alpha:
    print('Конверсия в покупку не увеличилась')
else:
    print('Конверсия в покупку увеличилась')

```

Размер группы A - 5383, размер группы B - 5467, покупок в группе A - 1542, покупок в группе B - 1687

Предпосылка о достаточном количестве данных выполняется!

pvalue=0.005870328999063137

stat_ztest=-2.519844825299311

Конверсия в покупку увеличилась

На основании проведенного теста можно сделать следующие выводы:

- данных для оценки результатов теста достаточно, количество пользователей в группах больше рассчитанного минимального значения выборки;

- полученное значение p_value меньше установленного уровня значимости теста, а значит можно сказать, что нулевая гипотеза о том, что конверсия не увеличилась, отвергается;
- результаты показали увеличение конверсии в покупку в группе В на 2,51% больше, чем в группе А;
- новый интерфейс действительно можно считать более удобным для пользователей, он увеличивает конверсию в покупку на ожидаемые значения. Можно рассмотреть предложение о раскатывании нового интерфейса на всех пользователей.